
VIBESWEBENCH: Can AI Co-Worker Agents Do Real-World Software Engineering by Vibe Coding?

Anonymous Authors¹

Abstract

The growing reliance on autonomous agents for real-world software engineering demands evaluation that reflects actual workflows, where developers give feedback, agents recover from mistakes, and tasks span the full project lifecycle. Existing benchmarks, however, focus largely on single-turn code generation from pristine prompts, ignoring the iterative reality of human-agent collaboration. To address this gap, we introduce **VIBESWEBENCH**, the first benchmark to evaluate AI co-worker agents on the four core capabilities of real-world development: greenfield construction, feature evolution, specification-driven bug repair, and database operations—and on the multi-turn user-corrective interactions that authentic workflows actually produce. **VIBESWEBENCH** contains 109 tasks (cumulative weight 230) across two real backend projects, evaluated against a comprehensive human-validated test suites with no LLM-as-a-judge component. To capture multi-turn recovery without proxy-developer confounds, we introduce *Trajectory Unrolling*, a procedure that converts authentic human-agent failure sessions into independent single-turn tasks while preserving human-authored corrective signal. Across four frontier coding agents—Codex, Claude Code, Gemini Code Assist, and Terminus-2—the strongest configuration (Codex with `gpt-5.5`) achieves only 20.87% overall, with performance dropping from 36.11% on bug repair to 2.56% on greenfield, and no successful completions on the hard subset. We release **VIBESWEBENCH**, its harness, and all artifacts to advance autonomous software engineering.

1. Introduction

As AI coding agents become increasingly capable of long-horizon autonomous execution, software development is undergoing a structural shift. In workflows now commonly

called *vibe coding* (Ray, 2025; Fawzy et al., 2025; Sapkota et al., 2025; Maes, 2025; Meske et al., 2025; Piskala, 2026), a human specifies intent—what a system should do, how it should behave, and how it should evolve—while an AI agent autonomously implements, self-validate, and iteratively refines the system through corrective feedback. This paradigm has become practical through a new generation of command-line coding agents, including **Claude Code**, **Codex**, **Cursor**, **Gemini Code Assist**, and **GitHub Copilot**. Increasingly, developers act less as direct implementers and more as supervisors who specify intent, validate outputs, and provide corrective guidance.

This transformation creates a fundamental evaluation challenge: *what capabilities must AI co-worker agents possess to reliably build production-grade software, and how should those capabilities be measured?* Existing coding benchmarks answer this question only partially because each evaluates a single narrow slice of software engineering. NL2Repo-Bench (Ding et al., 2025), ProjectEval (Liu et al., 2025), ProjDevBench (Lu et al., 2026), and Terminal-Bench (Merrill et al., 2026) primarily focus on greenfield implementation task; FEA-Bench (Li et al., 2025) evaluates repository-level feature evolution; and the SWE-Bench family (Jimenez et al., 2023; Yang et al., 2024; Deng et al., 2025; Wang et al., 2025; Zhang et al., 2025c) focuses on issue-driven bug fixing in existing repositories. Collectively, these benchmarks cover isolated capabilities, but not the full software lifecycle that vibe coding demands.

More importantly, existing evaluations miss a more fundamental dimension: *multi-turn corrective interaction*. In practice, autonomous coding agents rarely succeed in a single pass; they propose implementations, receive corrective feedback, revise their approach, and iteratively converge toward specification-compliant solutions. Whether an agent can recover from mistakes, interpret corrective guidance, and adapt across successive iterations is therefore central to its practical utility. Yet current benchmarks largely evaluate single-shot execution, leaving the community without a rigorous and reproducible framework for measuring this core capability.

To address this gap, we introduce **VIBESWEBENCH**, the first benchmark designed specifically for evaluating AI co-

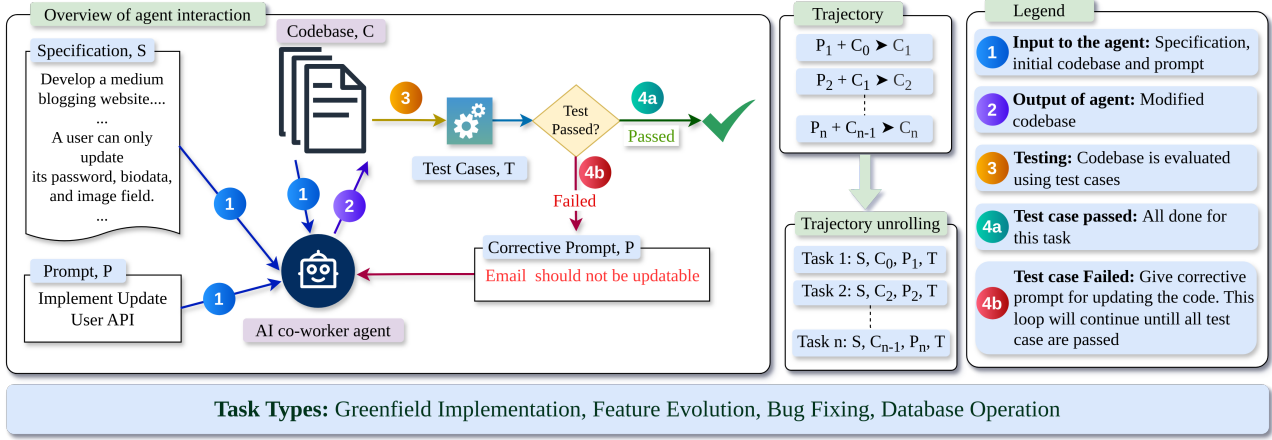


Figure 1. Workflow of **VIBESWEBENCH** creation. An agent receives a specification, codebase, and implementation prompt (1), produces an updated repository (2), and is evaluated by deterministic tests (3). Success terminates the trajectory (4a); failure triggers human corrective feedback (4b). The resulting authentic multi-turn trajectories are unrolled into the tasks that form **VIBESWEBENCH**.

worker agents in realistic vibe-coding workflows. The workflow is illustrated in Figure 1. A user first provides a specification and implementation request, after which the agent generates or modifies the codebase and returns an updated repository. If the implementation fails to satisfy the specification, the user provides corrective feedback, and the interaction continues iteratively until the codebase converges to a compliant state. **VIBESWEBENCH** evaluates both the implementation capabilities required for real software engineering and the corrective interaction loop that governs practical deployment. All tasks are articulated from scratch in realistic development scenarios rather than sourced from existing repositories (e.g., GitHub), avoiding contamination from memorized code or repository overlap. Following best practices from works such as Terminal-Bench (Merrill et al., 2026) and NL2Repo-Bench (Ding et al., 2025), we employ precise, unambiguous specifications to ensure deterministic evaluation — a necessary condition for reproducible benchmark.

Our first contribution is **VIBESWEBENCH**: 109 specification-anchored tasks (cumulative weight 230) across two real backend projects, curated based on three real developers interactions fully from scratch without any detailed specifications with CoPilot and Codex over a month (200 hours+), evaluated against deterministic, human-validated test suites with no LLM-as-a-judge component (Gu et al., 2025; He et al., 2026). The benchmark spans four implementation capabilities—greenfield construction, feature evolution, bug repair, and database operations—together with multi-turn corrective interaction. Each task is scored binary (pass only if all tests pass), and the overall score is the complexity-weighted average of successful tasks. Section 4.2 verifies leaderboard robustness under three weighting schemes.

Our second contribution is *Trajectory unrolling*, a method-

ology for evaluating multi-turn corrective interaction—a central yet previously underexplored capability in vibe coding workflows. The challenge is uniquely hard: unlike domains where a proxy LLM can plausibly stand in for a human user, corrective feedback in vibe coding must be anchored in a precise specification, and any proxy therefore conflates its own dynamic specification-interpretation errors with the evaluated agent’s failures; yet replaying full trajectories is expensive, stateful, and non-deterministic, while collapsing to single-shot evaluation discards the very recovery loop that defines real practice. Trajectory unrolling resolves this trilemma by decomposing authentic human-agent failure trajectories into a sequence of independent, specification-anchored single-turn tasks, each paired with the original human-authored corrective prompt—preserving multi-turn realism, eliminating the proxy-developer confound, and restoring deterministic reproducibility. The methodology is benchmark-agnostic and generalizes beyond **VIBESWEBENCH** to any future evaluation of iterative human-AI software development.

Our third contribution is an empirical capability map. Across four frontier coding agents—Codex CLI, Claude Code CLI, Gemini Code Assist CLI, and Terminus-2 (Merrill et al., 2026)—the strongest configuration (Codex CLI with gpt-5.5) achieves only 20.87% overall, with performance collapsing from 36.11% on bug repair to 2.56% on greenfield construction, and zero success on the hard 8-task subset. Figure 5 of Appendix further shows that **VIBESWEBENCH** remains far from saturation for current agents.

We release **VIBESWEBENCH**, a datasheet, a Croissant (Akhtar et al., 2024) metadata file, and reproducibility scripts under a permissive license, anonymously hosted at this url.

Table 1. Feature comparison across existing benchmarks and VIBESWEBENCH. Each column corresponds to one capability required of an agent operating under Spec-Driven Development. SWE-Bench Multimodal (Yang et al., 2024) accepts image inputs for issue reproduction but does not evaluate front-end visual fidelity against specifications.

Benchmark	Greenfield Implementation	Feature Evolution	Bug Fixing	DB Operations	User Interaction
Terminal-Bench (Merrill et al., 2026)	✓	–	–	–	–
NL2Repo-Bench (Ding et al., 2025)	✓	–	–	–	–
Vibe Code Bench (Tran et al., 2026)	✓	–	–	–	–
ProjectEval (Liu et al., 2025)	✓	–	–	–	–
ProjDevBench (Lu et al., 2026)	✓	–	–	–	–
FEA-Bench (Li et al., 2025)	–	✓	–	–	–
SWE-Bench (Jimenez et al., 2023)	–	–	✓	–	–
SWE-Bench Pro (Deng et al., 2025)	–	–	✓	–	–
SWE-Bench++ (Wang et al., 2025)	–	–	✓	–	–
SWE-bench-Live (Zhang et al., 2025c)	–	–	✓	–	–
VIBESWEBENCH (Ours)	✓	✓	✓	✓	✓

2. Related Work

The evaluation of LLMs for software engineering has progressed rapidly from isolated code synthesis tasks to autonomous agents operating over full repositories and realistic development environments. We organize prior work into three areas: coding benchmarks, autonomous coding agents, and spec-driven software engineering. VIBESWEBENCH sits at their intersection.

2.1. Coding Benchmarks

Function-level generation: Early benchmarks evaluate localized code synthesis in self-contained settings. HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) measure function generation from natural-language descriptions using hidden unit tests. Subsequent benchmarks broaden scope: ClassEval (Du et al., 2023) targets class-level generation with intra-class dependencies; xCodeEval (Khan et al., 2023) evaluates multilingual competitive-programming tasks; and LiveCodeBench (Jain et al., 2024) introduces a contamination-resistant stream of coding problems. While valuable for measuring reasoning and synthesis, these benchmarks abstract away repository structure, multi-file dependencies, environment setup, and long-horizon software evolution.

Repository-level evaluation: Repository-scale benchmarks introduce realistic software contexts. The SWE-Bench family (Jimenez et al., 2023; Deng et al., 2025; Wang et al., 2025; Yang et al., 2024; Miserendino et al., 2025; Xu et al., 2025) evaluates issue resolution in real GitHub repositories, establishing patch generation and bug fixing as the dominant evaluation paradigm. Complementary benchmarks explore adjacent capabilities: RepoBench (Liu et al., 2023) and M2RC-Eval (Liu et al., 2024) study repository-

level retrieval and completion; Commit0 (Zhao et al., 2024) evaluates library construction from scratch with structural priors; PaperBench (Starace et al., 2025) measures research-code reproduction; NL2Repo-Bench (Ding et al., 2025) evaluates long-horizon repository generation from natural-language specification; Vibe Code Bench (Tran et al., 2026) covers the full end-to-end process of creating a functional application from nothing; and Terminal-Bench (Merrill et al., 2026) stresses agent competence in realistic shell environments.

Despite their diversity, no existing benchmark is constructed around the holistic implementation demands of Spec-Driven Development. Concretely, an SDD-aligned agent must master four interrelated capabilities: (i) greenfield codebase generation from a specification; (ii) iterative feature evolution (addition, modification, deletion) in an existing codebase; (iii) specification-driven bug fixing; and (iv) practical database operations:schemas, queries, and migrations. No prior benchmark evaluates all four together. VIBESWEBENCH is designed to fill precisely this gap.

2.2. Autonomous Coding Agents

Recent advances in Agent–Computer Interfaces (ACIs) have transformed LLMs from passive code generators into autonomous software agents. SWE-agent (Yang et al., 2024) introduced a repository-level interface for search, navigation, editing, and test execution, enabling strong performance on software repair tasks. OpenHands (Wang et al., 2024) generalizes this paradigm into an open framework for end-to-end autonomous development with planning, execution, and self-validation. Commercial systems, including Claude Code, OpenAI Codex, Cursor, Gemini Code Assist, and GitHub Copilot, have brought autonomous coding into mainstream practice.

However, evaluation of these agents remains concentrated on edit-and-repair settings within existing repositories. Their ability to autonomously construct, evolve, repair, and manage specification-aligned software systems over long horizons remains largely unmeasured. **VIBESWEBENCH** directly targets this gap.

2.3. Spec-Driven Development

Spec-Driven Development (SDD) (Piskala, 2026) inverts conventional software engineering practice: a formal specification, rather than code, serves as the central source of truth. This mitigates “requirement rot” by establishing a shared specification against which both humans and automated systems can develop and validate software. SDD proceeds through three iterative phases—requirement evolution, autonomous implementation, and black-box validation (Qian et al., 2024; Hong et al., 2024)—making it a natural foundation for vibe coding workflows.

Adjacent works, including ArtifactsBench (Zhang et al., 2025b), AutoCodeBench (Chou et al., 2025), and CodeCriticBench (Zhang et al., 2025a), explore complementary aspects of specification-grounded development, but do not evaluate end-to-end autonomous implementation. In particular, none assess whether an agent can build, evolve, repair, and manage persistent state from a formal specification alone.

VIBESWEBENCH fills this gap. Each task provides a formal specification and requires an agent to build or modify the corresponding software system without human coding involvement, directly evaluating the implementation capabilities required under SDD. However, unlike previous works, we derived the specifications also via vibecoding the agents (Copilot, and Codex) during our data creations (See Section 3.4).

3. VIBESWEBENCH

Our goal is to evaluate whether LLM-based coding agents can translate formal specifications into correct, runnable codebases without human intervention. **VIBESWEBENCH** is built around three methodological ideas: (1) specification-anchored task design with weighted scoring for deterministic evaluation, (2) trajectory unrolling for reproducible assessment of multi-turn recovery, and (3) multi-dimension task categorization for fine-grained capability analysis. Key terminology used throughout the paper is defined in Appendix A.

3.1. Task Design

We have designed tasks around two real-life software projects, yielding a total of 109 tasks. Each project is anchored to a comprehensive *reference technical specification*

that defines API endpoints, socket events, and business logic. This specification serves a dual role: it is given to the agent as the authoritative input, and it also defines the ground truth against which the final implementation is evaluated. Grounding every task in a formal specification allows deterministic, reproducible, and automated evaluation without relying on LLM-as-a-judge metrics. Furthermore, each task is manually curated and validated to minimize any risk of contamination with existing LLM training data.

Given a well-defined specification, we decompose each project into tasks spanning different levels of granularity and complexity, mirroring the primary implementation scenarios encountered in real-world software development:

- **Greenfield implementation:** develops an entire backend system starting from an empty codebase.
- **Feature evolution:** introduces, modifies, or removes functionality within an existing codebase.
- **Bug fixing:** corrects faulty or non-compliant implementations to satisfy the target specification.
- **Database interaction:** implements database operations and associated persistence-layer logic.

Each task provides the agent with a well-defined portion (or the entirety) of the specification and requires it to produce a corresponding implementation, validated against a carefully constructed deterministic test suite. Tasks are defined with sufficient precision to support unambiguous, reproducible evaluation outcomes.

Because tasks vary significantly in scope and difficulty, a simple success-rate metric would be misleading: treating a single-feature task and a full-project implementation as equal units of work plainly misrepresents relative agent capability. We therefore assign each task i a positive weight $w_i > 0$ proportional to its estimated complexity and development effort. Each task receives a binary score s_i , where $s_i = 1$ if the agent passes all associated test cases and $s_i = 0$ otherwise. An agent’s overall **VIBESWEBENCH** score $\mathcal{S}$ is then the weighted average, $\mathcal{S} = \sum_{i=1}^N w_i s_i / \sum_{i=1}^N w_i$, where N is the total number of tasks in the benchmark. A score of $\mathcal{S} = 1$ indicates that the agent passed every test case across all tasks; $\mathcal{S} = 0$ indicates complete failure on all tasks. This formulation ensures that harder, higher-scope tasks contribute proportionally more to the final score, providing a calibrated and meaningful measure of agent capability.

To ensure robust and trustworthy evaluation, we constructed the dataset and execution environment with secure sandbox execution, standardized containerized configurations, and a reproducible testing harness across all tasks, leveraging the Harbor framework (Harbor Framework Team, 2026).

Table 2. Dataset statistics for **VIBESWEBENCH**. *Count* is the number of tasks; *Weight* is the sum of task weights, reflecting scope and complexity). Total: 109 tasks, cumulative weight 230.

Dimension	Value	Count	Count %	Weight	Weight %
Project	Digital Office	83	76.1%	186	80.9%
	Real World	26	23.9%	44	19.1%
Interaction	Multi-turn	74	67.9%	108	47.0%
	Single-turn	35	32.1%	122	53.0%
Difficulty	Easy	48	44.0%	48	20.9%
	Medium	53	48.6%	128	55.7%
	Hard	8	7.3%	54	23.5%
Task Type	Bug Fixing	74	67.9%	108	47.0%
	Feature Evolution	26	23.9%	83	36.1%
	Greenfield	9	8.3%	39	17.0%
	DB Interaction	109	100%	230	100%

3.2. Corrective Tasks and Trajectory Unrolling

A well-written specification does not guarantee perfect single-turn execution. For long-horizon tasks, or when an agent is limited by reasoning, implementation, or planning failures, a single response may misinterpret requirements, produce incomplete functionality, introduce bugs, or break existing behavior. In realistic development workflows, users inspect the resulting codebase, identify deviations from the specification, and issue *corrective follow-up prompts* to steer the implementation back into alignment. To evaluate this recovery capability, we introduce *corrective tasks*, which capture two common scenarios:

1. **Multi-turn failure recovery:** An agent produces a flawed implementation, after which the user provides corrective instructions. We evaluate whether the agent can recover from this degraded state in a single subsequent turn.
2. **Codebase adoption:** A team with an existing, legacy, or partially developed codebase wishes to adopt an SDD-based vibe-coding workflow, but the current implementation does not satisfy the target specification and must first be brought into compliance.

In both settings, the agent is given a codebase that deviates from the reference specification and is tasked with producing a corrected version that passes all tests. The codebase-adoption setting is straightforward: we provide a non-compliant codebase together with the target specification and measure whether the agent can restore alignment.

Modeling multi-turn interaction: We formalize an extended development session as a state-transition process. Let C_0 denote the initial codebase. The user issues prompt P_1 , and the agent produces a new state C_1 . If C_1 fails to satisfy the specification, the user issues a corrective

prompt P_2 , producing C_2 . This iterative process follows: $C_n = \text{Agent}(C_{n-1}, P_n)$ (see Appendix B). The number of turns required to reach compliance is itself informative: stronger agents converge in fewer iterations (ideally $n = 1$), whereas weaker agents may require repeated correction—or fail to converge entirely—as accumulated edits progressively degrade the codebase, making recovery increasingly difficult.

The proxy-developer confound: A natural evaluation strategy is to simulate this process using an automated proxy developer: an LLM that generates corrective prompts P_i based on prior conversation history and test failures at state C_{i-1} . However, this introduces a fundamental confound by coupling two independent sources of error: the quality of the corrective prompt and the capability of the evaluated agent. When recovery fails, it becomes unclear whether the agent was incapable of repair or simply received poor guidance.

Trajectory Unrolling: **VIBESWEBENCH** resolves this issue through **Trajectory Unrolling** (also referred to as *trajectory flattening*). Instead of simulating corrective interactions with a proxy developer, we collect real multi-turn human-agent development sessions. From each trajectory, we extract intermediate codebase states together with the corresponding human-authored corrective prompts, and convert every step into an independent single-turn benchmark task. A complete example how trajectory unrolling works is illustrated in Appendix B.

Trajectory Unrolling eliminates the need for a proxy developer entirely. Because corrective prompts are authored by humans, evaluation remains deterministic, reproducible, and free from prompt-quality confounds. The method is benchmark-agnostic and can be adopted by any future evaluation that aims to measure multi-turn corrective interaction. At the same time, the resulting tasks preserve the essential

challenge of multi-turn development: interpreting corrective intent and restoring a degraded codebase to compliance with the specification.

Together, specification-anchored project decomposition and Trajectory Unrolling constitute the two structural pillars of **VIBESWEBENCH**. All tasks are manually curated and rigorously validated to ensure clarity, realism, and evaluation quality.

3.3. Multi-dimension Task Categorization

To enable fine-grained capability analysis, the 109 tasks (cumulative weight 230) in **VIBESWEBENCH** are organized along four orthogonal dimensions:

- **Project:** Tasks are grouped by their originating software project. Our dataset consists of two real-world backend projects. The first is *RealWorld*, a popular GitHub repository for demo applications. The second, *Digital Office*, is designed by us and includes functionalities such as employee management, real-time chat, and file transfer. We focus exclusively on backend development for both projects. In *RealWorld*, API documentation serves as the technical specification, whereas in *Digital Office*, the specification includes both API and WebSocket event documentation.
- **Interaction Type:** Tasks are categorized as *single-turn* (where the agent must succeed from a clean initial state) or *multi-turn* (where the agent must recover from a degraded state produced by Trajectory Unrolling). Both projects include tasks of each type.
- **Difficulty:** Tasks are labeled as *easy*, *medium*, or *hard*, reflecting the scope and complexity of the required implementation. Task weights w_i are calibrated accordingly.
- **Task Type:** Tasks are categorized into greenfield implementation, feature evolution (addition, modification, or deletion), specification-driven bug fixing, and database-intensive operations. These correspond to the four implementation capability axes introduced in Section 1.

Detailed statistics and distributions across these axes are presented in Table 2.

3.4. Data collection and contamination handling

To avoid any contamination, three real developers interacted with popular coding agents (Github Copilot, Codex, Claude Code, and Gemini Code Assist) to develop the two above mentioned project fully from scratch. Starting from intent, first the full API specific was derived, itself also via vibecoding and verified. Then development cycle started and vibecoded over a time frame of over 1 month (200 hours+). The trajectory was cumulated and process as above to form the high-quality tasks and dataset. The inadvertent inclusion of

our benchmark in future model training corpora poses an ongoing risk of data contamination. To help developers filter their training data, we follow the precedent set by Terminal-Bench (Merrill et al., 2026) and Big-Bench (Srivastava et al., 2023) by incorporating identifiable markers across all files in our public repository. Additionally, we withhold test cases and provide a public API for evaluating task correctness. This approach helps prevent contamination, at least with respect to the test set.

4. Experiments and Results

We evaluate **VIBESWEBENCH** with three frontier coding agents paired with five state-of-the-art language models: **Codex** (gpt-5.3-codex, gpt-5.5), **Claude Code** (claude-sonnet-4.6, claude-opus-4.7), and **Gemini Code Assist** (gemini-3.1-pro-preview, shortly gemini-3.1-pro). In addition, we assess an open-source agent, Terminus-2 (Merrill et al., 2026), paired with two smaller models: qwen3.6-plus and deepseek-v4-pro. For every task, the agent is executed in an isolated docker environment containing only the reference specification and the corresponding initial codebase state. The agent must satisfy the specification without human intervention, and the resulting codebase is scored exclusively by the hidden test suite.

Under this evaluation protocol, we report *Pass Rate* (the percentage of tasks for which all hidden tests pass), *Weighted Score* (S), *Input/Output Tokens* (in millions), and *Wall-clock Time* (in hours). All tasks are packaged with Harbor-compatible Docker definitions (Harbor Framework Team, 2026), making the evaluation fully reproducible; the dataset release also includes instructions for rerunning the experiments.

4.1. Main Results

Table 3 summarizes the overall performance across all evaluated systems. A key observation is that **VIBESWEBENCH** remains far from being solved by current frontier models. None of the evaluated configurations surpasses a 21% weighted score, indicating that strong performance on existing coding benchmarks does not generalize to specification-driven, long-horizon software engineering tasks.

Among the tested configurations, Codex with gpt-5.5 performs best overall, reaching a 36.7% pass rate and a 20.87% weighted score. Claude Code with claude-opus-4.7 ranks second at 19.13%, while Claude Code with claude-sonnet-4.6 and Codex with gpt-5.3-codex tie at 16.52%. Gemini Code Assist with gemini-3.1-pro attains a 15.22% weighted score. These margins are modest in absolute terms, but they consistently favor the strongest Codex configuration.

Table 3. Main results on **VIBESWEBENCH** (109 tasks, total weight 230). *Pass Rate* is the fraction of tasks on which all test cases pass. *Wtd. Score* is S that is defined in Section 3. Input and output tokens are reported in millions (M). Gray shaded row represent the best performance.

Agent	Model	Pass Rate	Wtd. Score	In/Out (M)	Time (h)
Codex	gpt-5.5	36.7%	20.87%	142.7 / 1.3	7.77
Claude Code	claude-opus-4.7	33.0%	19.13%	456.7 / 3.0	11.11
Codex	gpt-5.3-codex	30.3%	16.52%	174.1 / 2.0	7.84
Claude Code	claude-sonnet-4.6	29.4%	16.52%	190.3 / 2.8	11.66
Gemini Code	gemini-3.1-pro	27.5%	15.22%	244.6 / 2.0	14.64
Terminus-2	deepseek-v4-pro	19.3%	9.57%	111.4 / 3.9	37.45
Terminus-2	qwen3.6-plus	18.3%	9.13%	142.8 / 3.9	28.32

Efficiency differences are also substantial. The best-performing system, Codex with gpt-5.5, is not only the most accurate but also comparatively efficient, using 142.7M input tokens and finishing in 7.77 hours. By contrast, Claude Code with claude-opus-4.7 consumes 456.7M input tokens, and Gemini Code Assist records the longest runtime at 14.64 hours. Taken together, the results indicate that **VIBESWEBENCH** is challenging in both capability and efficiency dimensions.

In contrast, smaller models evaluated via Terminus-2 (Merrill et al., 2026) perform significantly worse. deepseek-v4-pro achieves a 19.3% pass rate and a 9.57% weighted score, while qwen3.6-plus reaches 18.3% and 9.13%, respectively. Despite moderate token usage, both configurations incur substantially higher wall-clock times, approximately 37 hours and 28 hours respectively. This latency is likely influenced by the deployment setup, as these models were accessed through OpenRouter rather than native inference endpoints, potentially introducing additional overhead.

To better contextualize these results, Appendix D provides three qualitative case study illustrating common near-miss and structural failure patterns observed in practice.

4.2. Ablation Study

To identify where current agents fail, we perform a detailed ablation of the strongest baseline, Codex with gpt-5.5, across the three structural axes of **VIBESWEBENCH**: task type, difficulty, and interaction type. Figure 2 summarizes the breakdown by task type and difficulty.

Performance by Task Type: As shown in Figure 2a (left), performance varies sharply by task type. We did not show database interaction because all the task has data interaction. The agent achieves a 36.11% weighted score on bug-fixing tasks, but drops to 9.64% on feature-evolution tasks and only 2.56% on greenfield tasks. This gap is meaningful. Bug fixing starts from a structurally coherent, runnable codebase that already exposes many of the relevant architectural decisions. Feature evolution and greenfield synthesis instead require the agent to infer missing structure, preserve con-

sistency across files, and implement new behavior directly from the specification. The contrast shows that localized repair is far easier for current agents than open-ended construction.

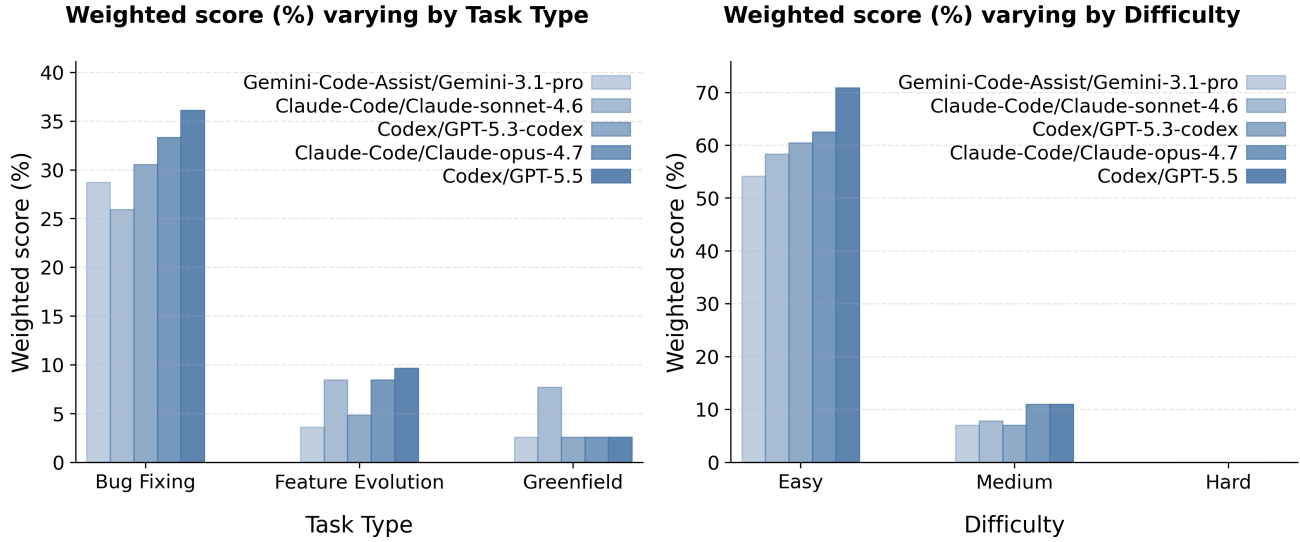
Performance by Difficulty: Tasks in **VIBESWEBENCH** are labeled easy, medium, or hard based on human judgments of scope and complexity. Figure 2b (right) shows a strongly non-linear performance drop: the agent achieves 70.83% weighted score on easy tasks, falls to 10.94% on medium tasks, and reaches zero success on hard tasks. Detailed analysis in Appendix C shows that author-assigned weights align closely with empirical difficulty derived from observed pass rates.

Performance by Interaction Type: A surprising pattern appears when we separate single-turn and multi-turn tasks: the gpt-5.5 baseline performs much better on multi-turn tasks (47.3% pass rate, 36.11% weighted score) than on single-turn tasks (14.3% pass rate, 7.38% weighted score). This inversion is a direct consequence of Trajectory Unrolling (Section 3). Multi-turn tasks correspond to later corrective stages of a development session, so the agent receives a narrow, localized instruction such as fixing a specific JWT validation bug. Single-turn tasks usually correspond to the initial, broader request, often starting from a blank or lightly scaffolded state. The result does not imply that multi-turn development is intrinsically easier; instead, it shows that current agents are far stronger at targeted correction than at unconstrained end-to-end construction.

4.3. Error Analysis and Improvement

To better understand the remaining gap, we manually analyzed the 69 zero-reward trials of the Codex (gpt-5.5) baseline. The dominant pattern is not catastrophic collapse, but near-miss behavior: the agent often produces code that is largely functional while still failing exact contract requirements or critical edge cases. In other words, many failures occur close to the target solution, but not close enough to pass a strict specification-grounded evaluation.

1. Specification Misreading / Contract Mismatch



(a) Performance comparison of different agents broken down by task type.

(b) Performance comparison of different agents broken down by difficulty level.

Figure 2. Comparison of agent performance across task type and difficulty.

(69.6%): The most common failure mode, accounting for 48 of 69 trials, occurs when the implementation is broadly plausible but violates a precise system contract. Typical examples: omitting required validation, missing audit-log side effects, or producing malformed WebSocket payloads. These runs often earn substantial partial credit, but they fail the benchmark’s exact contract checks.

- 2. Incomplete Implementation / Omitted Behavior (13.0%):** In 9 trials, the agent appears to understand the task but leaves required behavior unfinished. This pattern is especially common for WebSocket events, notifications, and audit features. These failures also have the highest average runtime (6.10 minutes), consistent with the agent exhausting its effective context or budget while attempting larger multi-file additions.
- 3. Regression / Collateral Breakage (10.1 %):** In 7 trials, the agent fixes the targeted issue or adds the requested feature, but breaks previously correct behavior elsewhere in the system. For example, repairing an administrative registration path can inadvertently break downstream login logic. This validates the need for full end-to-end test-suite evaluation rather than narrow task-local checks.
- 4. Runtime and Schema Failures (7.2 %):** The least frequent category, covering 5 trials, consists of hard backend failures such as SQL type mismatches, missing schema elements, or transport-level exceptions including `BrokenPipeError`. These produce outright server errors rather than near-correct behavior.

These failure modes suggest concrete directions for improving coding agents. The prevalence of contract mismatches

(69.6%) points to a need for specification-grounded self-verification – e.g., checklist-style validation of status codes, side effects, and schemas before finalizing a solution. Incomplete implementations (13.0%) call for better long-horizon planning and sub-goal decomposition, particularly for multi-file edits. Regression failures (10.1%) highlight the importance of running broader post-change test suites, not just task-local checks. Finally, runtime and schema errors (7.2%) indicate a need for stronger environment-aware reasoning about database migrations, type constraints, and service initialization. Overall, progress on **VIBESWEBENCH** depends less on raw generation and more on systematic specification tracking and self-verification.

5. Conclusion

The transition from manual programming to spec-driven vibe coding shifts the developer’s role from builder to director, demanding agents that can independently translate formal specifications into robust code. We introduced **VIBESWEBENCH**, the first unified framework for evaluating agents across the full implementation phase of Spec-Driven Development: greenfield synthesis, feature evolution, and bug fixing. Our experiments exposed a critical asymmetry: current state-of-the-art models excel at localized repairs but struggle profoundly with architectural construction, with performance plummeting from 71% on easy tasks to 0% on hard tasks and no agent exceeding 21% overall. As vibe coding accelerates, **VIBESWEBENCH** serves as both a diagnostic instrument for current agentic workflows and a concrete target for the next generation of autonomous software engineers. We release all code, data, and evaluation tools to support progress in this area.

References

- Akhtar, M., Benjelloun, O., Conforti, C., Gijsbers, P., Giner-Miguel, J., Jain, N., Kuchnik, M., Lhoest, Q., Marcenac, P., Maskey, M., Mattson, P., Oala, L., Ruysen, P., Shinde, R., Simperl, E., Thomas, G., Tykhonov, S., Vanschoren, J., van der Velde, J., Vogler, S., and Wu, C.-J. Croissant: A metadata format for ml-ready datasets. In *Proceedings of the Eighth Workshop on Data Management for End-to-End Machine Learning*, SIGMOD/PODS '24, pp. 1–6. ACM, 2024. doi: 10.1145/3650203.3663326. URL <http://dx.doi.org/10.1145/3650203.3663326>.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- Chou, J., Liu, A., Deng, Y., Zeng, Z., Zhang, T., Zhu, H., Cai, J., Mao, Y., Zhang, C., Tan, L., Xu, Z., Zhai, B., Liu, H., Zhu, S., Zhou, W., and Lian, F. Autocodebench: Large language models are automatic code benchmark generators, 2025. URL <https://arxiv.org/abs/2508.09101>.
- Deng, X., Da, J., Pan, E., He, Y. Y., Ide, C., Garg, K., Lauffer, N., Park, A., Pasari, N., Rane, C., et al. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.
- Ding, J., Long, S., Pu, C., Zhou, H., Gao, H., Gao, X., He, C., Hou, Y., Hu, F., Li, Z., et al. Nl2repo-bench: Towards long-horizon repository generation evaluation of coding agents. *arXiv preprint arXiv:2512.12730*, 2025.
- Du, X., Liu, M., Wang, K., Wang, H., Liu, J., Chen, Y., Feng, J., Sha, C., Peng, X., and Lou, Y. Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation, 2023. URL <https://arxiv.org/abs/2308.01861>.
- Fawzy, A., Tahir, A., and Blincoe, K. Vibe coding in practice: Motivations, challenges, and a future outlook—a grey literature review. *arXiv preprint arXiv:2510.00328*, 2025.
- Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., Wang, S., Zhang, K., Wang, Y., Gao, W., Ni, L., and Guo, J. A survey on llm-as-a-judge, 2025. URL <https://arxiv.org/abs/2411.15594>.
- Harbor Framework Team. Harbor: A framework for evaluating and optimizing agents and models in container environments, January 2026. URL <https://github.com/harbor-framework/harbor>.
- He, J., Shi, J., Zhuo, T. Y., Treude, C., Sun, J., Xing, Z., Du, X., and Lo, D. Llm-as-a-judge for software engineering: Literature review, vision, and the road ahead. *ACM Trans. Softw. Eng. Methodol.*, February 2026. ISSN 1049-331X. doi: 10.1145/3797276. URL <https://doi.org/10.1145/3797276>. Just Accepted.
- Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., and Schmidhuber, J. Metagpt: Meta programming for a multi-agent collaborative framework, 2024. URL <https://arxiv.org/abs/2308.00352>.
- Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., and Stoica, I. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL <https://arxiv.org/abs/2403.07974>.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.
- Khan, M. A. M., Bari, M. S., Do, X. L., Wang, W., Parvez, M. R., and Joty, S. xcodeeval: A large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval, 2023. URL <https://arxiv.org/abs/2303.03004>.
- Li, W., Zhang, X., Guo, Z., Mao, S., Luo, W., Peng, G., Huang, Y., Wang, H., and Li, S. Fea-bench: A benchmark for evaluating repository-level code generation for feature implementation, 2025. URL <https://arxiv.org/abs/2503.06680>.
- Liu, J., Deng, K., Liu, C., Yang, J., Liu, S., Zhu, H., Zhao, P., Chai, L., Wu, Y., Jin, K., Zhang, G., Wang, Z., Zhang, G., Xiang, B., Su, W., and Zheng, B. M2rc-eval: Massively multilingual repository-level code completion evaluation, 2024. URL <https://arxiv.org/abs/2410.21157>.
- Liu, K., Pan, Y., Xiang, Y., He, D., Li, J., Du, Y., and Gao, T. Projecteval: A benchmark for programming agents automated evaluation on project-level code generation, 2025. URL <https://arxiv.org/abs/2503.07010>.
- Liu, T., Xu, C., and McAuley, J. Repobench: Benchmarking repository-level code auto-completion systems, 2023. URL <https://arxiv.org/abs/2306.03091>.
- Lu, P., Zhang, S., Hou, Y., Ye, L., Huang, C., Chen, Z., Zeng, J., Jiang, H., Liu, P., Wang, Y., and Yang, M.-H. Projdevbench: Benchmarking ai coding agents on end-to-end project development, 2026. URL <https://arxiv.org/abs/2602.01655>.
- Maes, S. H. The gotchas of ai coding and vibe coding. it's all about support and maintenance. *OSF Preprints*, 2025.
- Merrill, M. A., Shaw, A. G., Carlini, N., Li, B., Raj, H., Bercovich, I., Shi, L., Shin, J. Y., Walshe, T., Buchanan, E. K., et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.
- Meske, C., Hermanns, T., Von der Weiden, E., Loser, K.-U., and Berger, T. Vibe coding as a reconfiguration of intent mediation in software development: Definition, implications, and research agenda. *IEEE Access*, 13:213242–213259, 2025.

- Miserendino, S., Wang, M., Patwardhan, T., and Heidecke, J. Swe-lancer: Can frontier llms earn \$1 million from real-world free-lance software engineering? *arXiv preprint arXiv:2502.12115*, 2025.
- Piskala, D. B. Spec-driven development: From code to contract in the age of ai coding assistants. *arXiv preprint arXiv:2602.00180*, 2026.
- Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., and Sun, M. Chatdev: Communicative agents for software development, 2024. URL <https://arxiv.org/abs/2307.07924>.
- Ray, P. P. A review on vibe coding: Fundamentals, state-of-the-art, challenges and future directions. *Authorea Preprints*, 2025.
- Sapkota, R., Roumeliotis, K. I., and Karkee, M. Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic ai. *arXiv preprint arXiv:2505.19443*, 2025.
- Srivastava, A., Rastogi, A., Rao, A., Shueb, A. A. M., ..., and Wu, Z. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models, 2023. URL <https://arxiv.org/abs/2206.04615>.
- Starace, G., Jaffe, O., Sherburn, D., Aung, J., Chan, J. S., Maksin, L., Dias, R., Mays, E., Kinsella, B., Thompson, W., Heidecke, J., Glaese, A., and Patwardhan, T. Paperbench: Evaluating ai’s ability to replicate ai research, 2025. URL <https://arxiv.org/abs/2504.01848>.
- Tran, H., Nashold, L., Krishnan, R., Bigeard, A., and Gu, A. Vibe code bench: Evaluating ai models on end-to-end web application development. In *ACM Conference on AI and Agentic Systems (ACM CAIS ’26)*, 2026. doi: 10.1145/3786335.3813180. URL <https://arxiv.org/abs/2603.04601>.
- Wang, L., Ramalho, L., Celestino, A., Pham, P. A., Liu, Y., Sinha, U. K., Portillo, A., Osunwa, O., and Maduekwe, G. Swe-bench++: A framework for the scalable generation of software engineering benchmarks from open-source repositories. *arXiv preprint arXiv:2512.17419*, 2025.
- Wang, X., Li, B., Song, Y., Xu, F. F., Tang, X., Zhuge, M., Pan, J., Song, Y., Li, B., Singh, J., et al. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- Xu, J., Deng, K., Li, W., Yu, S., Tang, H., Huang, H., Lai, Z., Zhan, Z., Wu, Y., Zhang, C., Lei, K., Yao, Y., Lei, X., Zhu, W., Feng, Z., Li, H., Xiong, J., Li, D., Gao, Z., Wu, K., Xiang, W., Zhan, Z., Zhang, Y., Gong, W., Gao, Z., Wang, G., Xue, Y., Li, M., Xie, M., Zhang, X., Wang, J., Zhuang, W., Lin, Z., Wang, H., Zhang, Z., Zhang, Y., Zhang, H., Chen, B., and Liu, J. Swe-compass: Towards unified evaluation of agentic coding abilities for large language models, 2025. URL <https://arxiv.org/abs/2511.05459>.
- Yang, J., Jimenez, C. E., Zhang, A. L., Lieret, K., Yang, J., Wu, X., Press, O., Muennighoff, N., Synnaeve, G., Narasimhan, K. R., et al. Swe-bench multimodal: Do ai systems generalize to visual software domains? *arXiv preprint arXiv:2410.03859*, 2024.
- Zhang, A., Dong, M., Liu, J., Zhang, W., Wang, Y., Yang, J., Zhang, G., Liu, T., Peng, Z., Tan, Y., Zhang, Y., Wang, Z., Wang, W., He, Y., Deng, K., Zhou, W., Huang, W., and Zhang, Z. Codecriticbench: A holistic code critique benchmark for large language models, 2025a. URL <https://arxiv.org/abs/2502.16614>.
- Zhang, C., Li, Y., Xu, C., Liu, J., Liu, A., Zhou, C., Deng, K., Wu, D., Huang, G., Li, K., Yi, Q., Xiong, R., Hu, S., Zhang, Y., Jiang, Y., Xu, Z., Zhang, Y., Zhou, W., Zhou, C., and Lian, F. Artifactsbench: Bridging the visual-interactive gap in llm code generation evaluation, 2025b. URL <https://arxiv.org/abs/2507.04952>.
- Zhang, L., He, S., Zhang, C., Kang, Y., Li, B., Xie, C., Wang, J., Wang, M., Huang, Y., Fu, S., Nallipogu, E., Lin, Q., Dang, Y., Rajmohan, S., and Zhang, D. Swe-bench goes live!, 2025c. URL <https://arxiv.org/abs/2505.23419>.
- Zhao, W., Jiang, N., Lee, C., Chiu, J. T., Cardie, C., Gallé, M., and Rush, A. M. Commit0: Library generation from scratch, 2024. URL <https://arxiv.org/abs/2412.01769>.

Appendix

A. Terminology

To ensure clarity, we first define the key terms used throughout this paper.

User: A developer or novice who adopts an SDD-based vibe-coding workflow. We assume the user has a clear understanding of the desired application and possesses the ability to verify the final implementation against the specification, but has no obligation to write or review source code directly.

Agent: Throughout this work, the term *agent* refers exclusively to an autonomous vibe-coding agent powered by an LLM—a system capable of generating or modifying codebases based on natural-language instructions without human involvement in the implementation step.

Conversation: A conversation is the sequence of messages exchanged between a user and an agent. A *single-turn* conversation contains exactly one user message and one agent response; a *multi-turn* conversation involves multiple alternating messages and responses constituting an iterative refinement session.

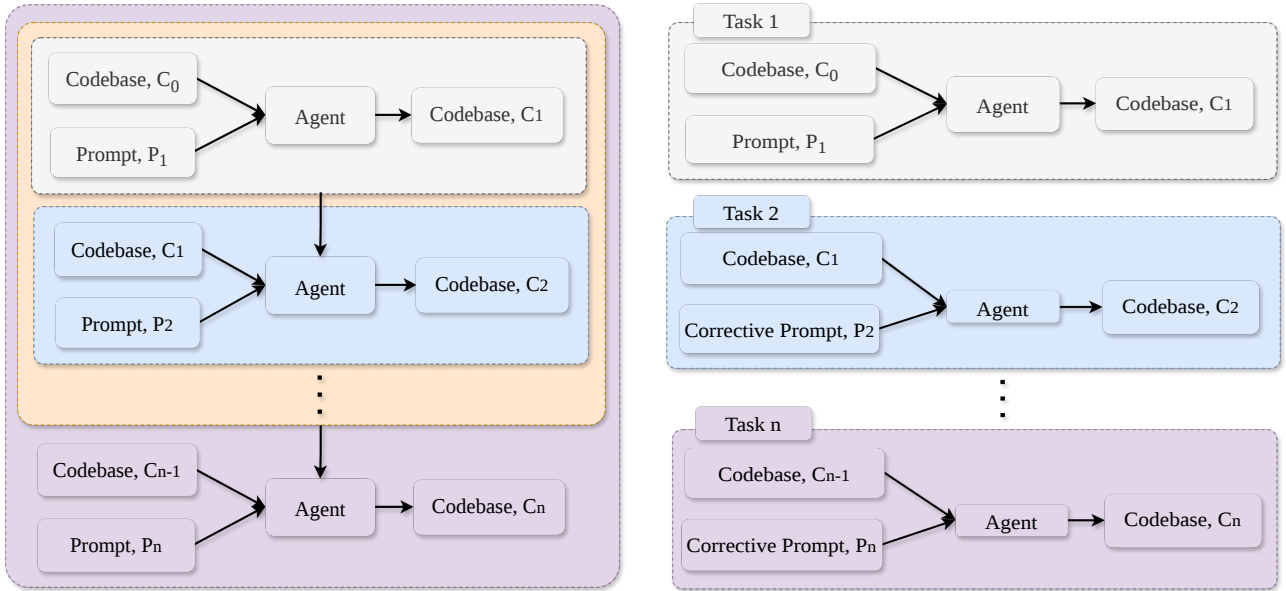
Task: A task is a self-contained unit of work that an agent is expected to complete within a single conversation.

Project: A project refers to a complete backend software package, including API logic, business rules, and database schema. While real-world applications commonly include a frontend, VIBESWEBENCH focuses exclusively on the backend component, as frontend evaluation requires visual or UI fidelity comparison against a reference interface—a direction we defer to future work.

Dataset: The dataset is a curated collection of tasks spanning multiple projects.

B. Example of How Trajectory Unrolling Works

For a multi-turn failure trajectory that reaches compliance in $n = 4$ turns, we construct four tasks:



(a) Multi-turn conversation modeled as a codebase state transition system.

(b) A multi-turn conversation decomposed into independent single-turn evaluation tasks.

Figure 3. Multi-turn conversation and trajectory unrolling.

- **Task 1:** Start from the clean initial state C_0 with prompt P_1 , evaluating from-scratch (greenfield) synthesis.
- **Task 2:** Start from degraded state C_1 with corrective prompt P_2 , evaluating recovery from a partially incorrect implementation.
- **Task 3:** Start from further-degraded state C_2 with prompt P_3 , evaluating repair after multiple rounds of potentially erroneous edits.
- **Task 4:** Start from last degraded state C_3 with prompt P_4 , evaluating repair after three rounds of potentially erroneous edits.

This process is illustrated in Figure 3b.

C. Predicted vs. Empirical Difficulty

The benchmark breakdowns support the interpretation that the author-assigned task weights are well aligned with task difficulty and implementation complexity. Figure 4 provides two complementary perspectives on this relationship by comparing model performance across both author-assigned weight levels and manually categorized difficulty levels.

Two independent views of the evaluation results exhibit the same qualitative trend:

- **Weight breakdown:** lower-weight tasks are solved substantially more often than higher-weight tasks. Figure 4a shows a clear decline in pass rates as task weights increase. Although there is minor variation between weight-2 and weight-3 tasks, the deviation is not sufficiently large to undermine the overall correspondence between predicted and empirical difficulty.
- **Difficulty breakdown:** easier tasks are solved substantially more often than medium and hard tasks. As shown in Figure 4b, model performance consistently decreases as task difficulty increases.

Taken together, these results suggest that higher author-assigned weights consistently correspond to tasks that are more difficult for current models to solve. Consequently, the weighting scheme appears to provide a meaningful proxy for the underlying implementation difficulty and complexity of benchmark tasks.



(a) Performance comparison of different agents grouped by task weight.

(b) Performance comparison of different agents grouped by task difficulty.

Figure 4. Comparison of agent performance across weight levels and difficulty categories. Results for tasks with weights greater than 3 are omitted because all evaluated agents achieved a 0.0% pass rate on those tasks.

D. Qualitative Example

D.1. Feature-evolution example: `task-rw-3`

This section presents a qualitative example from the benchmark using `task-rw-3`, a medium-difficulty RealWorld backend task. The example is useful because it isolates a narrow but realistic slice of backend development: an agent must extend an existing Node.js/Express codebase with authentication endpoints, create the required PostgreSQL schema, and satisfy a mix of API-contract, validation, and state-management requirements.

Importantly, the task is not a synthetic toy problem. It reflects a common real-world workflow in which a developer incrementally implements authentication functionality on top of an existing service skeleton. The agent must interpret the specification, map it to the existing codebase structure, and ensure that all components—including routing, validation, persistence, and token handling—work together correctly. This makes the task a compact but representative instance of specification-driven backend development under realistic constraints. We begin by presenting the exact instruction provided to the agent.

Input given to the agent

Task

Continue the backend implementation using **Node.js** and **Express**.

Refer to the complete technical specification in the `api.md` file. For this task, implement the following endpoints:

- **POST** `/api/users/login`
- **POST** `/api/users`
- **GET** `/api/user`
- **PUT** `/api/user`

DB

Create the required PostgreSQL tables for user data. Use the database connection configuration provided in `/app/config/db.js`.

Additional requirements

- Follow standard backend development practices.

The instruction above specifies the endpoints at a high level and directs the agent to consult `api.md` during implementation. Since `api.md` is a large file, we extract and present below the definitions of the relevant APIs. These definitions specify the exact input structure, authentication requirements, and permissible operations that the agent must satisfy to pass the hidden test suite.

Definition of APIs used in this task from `api.md`

1. **POST** `/api/users/login`

Description: Authenticate an existing user and return a JWT token.

Auth: Not required

Request:

- `email` (required)
- `password` (required)

2. **POST** `/api/users`

Description: Register a new user.

Auth: Not required

Request:

- `username` (required)

- email (required)
- password (required)

3. GET /api/user

Description: Retrieve the currently authenticated user.

Auth: Required (Bearer token)

4. PUT /api/user

Description: Update the current user's profile.

Auth: Required (Bearer token)

Accepted fields:

- password
- bio
- image

Expected output and what the tests check: The hidden verifier contains seven tests. Four important test cases are discussed below:

- **User registration:** POST /api/users must accept payloads of the form {"user": {"username": ..., "email": ..., "password": ...}}, create a user record, and return a token. Duplicate usernames, duplicate emails, and missing fields must be rejected.
- **Login:** POST /api/users/login must accept valid credentials and return the authenticated user with a token. Incorrect passwords, non-existent users, malformed payloads, empty payloads, and SQL-injection-style strings must fail.
- **Current user retrieval:** GET /api/user must require authentication, decode a valid token, and return the correct username and email. Invalid or missing tokens must be rejected.
- **User update:** PUT /api/user may update bio, image, and password. However, it must reject attempts to update username or email. An especially important edge case is that after a password-only update, the old token must still remain valid. This test therefore probes both field-level authorization and token semantics.

A representative hidden assertion is that a request attempting to update only the username must *fail*, rather than succeed silently or partially. This makes the task deceptively hard: a model can implement most of the endpoint successfully and still fail because it does not enforce one contract-level restriction.

How the five evaluated agent/model pairs perform. Table 4 summarizes the outcomes observed in the merged-job verifier logs for the five evaluated agent/model pairs.

The five runs naturally cluster into two qualitatively distinct failure modes.

Failure mode 1: near-complete implementations that miss one contract constraint: codex/gpt-5.3-codex, codex/gpt-5.5, and gemini-cli/gemini-3.1-pro-preview all solve most of the task. In the verifier logs, each of these systems passes registration, login, current-user retrieval, and most of the update flow. Their only failing test is the restriction that username updates must be rejected. This is a good example of a *specification-misreading* failure: the models build a largely functional authentication stack, yet they do not enforce one explicit business rule. In other words, the implementation is close to correct but not contract-compliant.

This pattern is especially revealing because the error is subtle. A developer using only superficial manual testing could easily conclude that the feature works: the database exists, users can register and log in, tokens are issued, and profile updates mostly behave correctly. The hidden tests expose that the code still violates an important immutability constraint. This is exactly the kind of near-miss that a specification-grounded benchmark is intended to detect.

Failure mode 2: route-level implementation without schema realization: `claude-code/claude-sonnet-4.6` and `claude-code/claude-opus-4.7` exhibit a different failure pattern. Both preserve the repository structure and the pre-existing smoke tests, but registration fails immediately with a database exception because the `users` table is never created. As a consequence, registration, login, current-user retrieval, and profile update all fail transitively. This is a *schema omission* failure rather than a business-logic failure: the models appear to understand the endpoint surface, but they do not complete the underlying persistence layer that the task explicitly requires.

This distinction matters for benchmark design. The same task simultaneously checks HTTP behavior, validation logic, authentication flow, and schema creation. A model that only edits Express routes without handling database initialization cannot succeed, even if the route signatures look plausible. The example therefore illustrates why database-backed tasks are an important component of realistic coding-agent evaluation.

What this example shows about the dataset: `task-rw-3` is a compact illustration of why the benchmark is challenging. First, it is *incremental*: the agent is not writing a full application from scratch, but it must extend an existing codebase without breaking the original skeleton. Second, it is *stateful*: correctness depends on interactions across multiple requests, including token issuance and reuse after password changes. Third, it is *database-backed*: the agent must create and query persistent state correctly. Finally, it is *specification-sensitive*: even models that are almost correct can fail because they miss a single prohibited update path.

Taken together, the results on this task show that current frontier coding models often reach the vicinity of the correct solution, but they fail in different ways depending on which layer of the specification they neglect. Some miss a narrow contract-level rule; others fail to realize the required schema entirely. This combination of near-miss behavior and heterogeneous failure modes makes `task-rw-3` a strong qualitative example of the broader benchmark design.

D.2. Greenfield-style implementation example: `task-rw-9`

For a contrasting case, we consider `task-rw-9`, a *greenfield-implementation* task. Unlike `task-rw-3`, which extends an existing backend slice, `task-rw-9` asks the agent to create a new RealWorld-style backend project from scratch, establish the standard folder structure, configure the database and CORS, and implement both authentication and profile-management functionality together with auditing. This makes it a clean example of specification-driven project bootstrapping rather than incremental editing.

Input given to the agent

Task

Create a new backend project using **Node.js** and **Express**, following a standard and maintainable backend architecture.

For this task, create the base project structure and implement the following endpoints:

Table 4. Performance of five agent/model pairs on `task-rw-3`. “Passed tests” is computed directly from the verifier logs for the seven hidden tests.

Agent/model	Passed tests	Primary failure mode
codex/ gpt-5.3-codex	6/7	Accepts a forbidden username update in PUT <code>/api/user</code> instead of rejecting the request.
codex/ gpt-5.5	6/7	Same contract violation as above: the update endpoint returns success for an attempted username change.
gemini-cli/ gemini-3.1-pro-preview	6/7	Same high-level error, but more severe: it accepts the username update and returns the modified username in the response.
claude-code/ claude-sonnet-4.6	3/7	Fails to create the required <code>users</code> relation, causing registration to crash with relation <code>"users"</code> does not exist.
claude-code/ claude-opus-4.7	3/7	Same schema-level failure as Sonnet: the authentication routes exist, but the database table is missing.

- POST /api/users/login
- POST /api/users
- GET /api/user
- PUT /api/user
- GET /api/profiles/:username
- POST /api/profiles/:username/follow
- DELETE /api/profiles/:username/follow
- GET /api/audits
- GET /hello

DB

- Create all required PostgreSQL tables and supporting database structures so the backend works correctly as soon as the server starts.

Additional requirements

- Configure CORS.
- Any unknown route must return a **404 not found response**.
- Define all required dependencies in `package.json`.
- Implement proper auditing.
- Follow standard backend development practices.

As with the other RealWorld tasks, the instruction is intentionally high level, and the hidden tests operationalize the true contract. The relevant portions of `api.md` combine the earlier authentication endpoints with profile-management and audit behavior.

Key contract requirements exercised by the verifier

1. **Project scaffold and HTTP basics:** the repository must contain the standard backend directories `controllers`, `models`, `routes`, `services`, `utils`, and `config`; `GET /hello` must return `hello world`; unknown routes must return `404`; and CORS headers must be present.
2. **Authentication endpoints:** registration, login, current-user retrieval, and user update must follow the same payload and authorization contracts as in `task-rw-3`, including the restriction that `username` updates are forbidden.
3. **Profile flows:** `GET /api/profiles/:username` is authentication-optional, while `follow` and `unfollow` are authentication-required and must correctly handle invalid tokens, non-existent users, and self-follow/self-unfollow attempts.
4. **Auditing:** `GET /api/audits` must expose the authenticated user's audit log, and the verifier checks not only whether routes succeed or fail, but also whether audit counts change correctly for successful operations and selected failed operations.

Expected output and what the tests check: The hidden verifier contains eight tests. Four are especially informative:

- **Repository and HTTP bootstrap:** the verifier checks the required folder structure, the `/hello` smoke route, the presence of CORS headers, and correct 404 behavior for unknown paths.
- **Authentication core:** it checks registration, login, current-user lookup, password updates, and the immutability of username and email in `PUT /api/user`.
- **Profile relationships:** it checks profile retrieval, follow/unfollow transitions, non-existent target handling, invalid-token behavior, and self-follow/self-unfollow rejection.
- **Auditing semantics:** it checks whether successful and failed operations generate audit records exactly where the contract expects them, which is unusually strict for a greenfield task and therefore hard to satisfy accidentally.

Although the surface area is much smaller than `task-do-9`, the task is still challenging because it combines project bootstrapping, endpoint implementation, and audit instrumentation in one greenfield slice. A model can build several correct routes and still fail if the scaffold, CORS, or audit semantics are incomplete.

How the five evaluated agent/model pairs perform. Table 5 summarizes the merged-job verifier outcomes.

Table 5. Performance of five agent/model pairs on `task-rw-9`. “Passed tests” comes directly from the verifier logs for the eight hidden tests.

Agent/model	Passed tests	Primary failure mode
codex/ gpt-5.3-codex	3/8	Builds much of the authentication core, but omits the required scaffold and CORS headers, still allows forbidden username updates, and mishandles self-unfollow.
claude-code/ claude-sonnet-4.6	2/8	Misses the required project scaffold and leaves registration crashing with an internal server error, so almost the entire backend remains unusable.
claude-code/ claude-opus-4.7	2/8	Follows the same pattern as Sonnet: <code>/hello</code> and 404 work, but the scaffold is incomplete and registration fails immediately with a server error.
codex/ gpt-5.5	2/8	Implements several routes, but auditing is systematically incomplete, causing failures across registration, login, current-user retrieval, updates, and profile actions.
gemini-cli/ gemini-3.1-pro-preview	2/8	Shows a similar audit-layer failure pattern to codex/ gpt-5.5, while also missing the required scaffold.

The five runs cluster into three qualitatively distinct failure profiles.

Failure mode 1: skeleton omitted before logic can even be exercised: `claude-code/claude-sonnet-4.6` and `claude-code/claude-opus-4.7` both pass only the trivial HTTP smoke checks. Their outputs suggest that they can stand up a minimal Express app, but they do not realize the required project structure and never produce a working registration path. This is a classic greenfield failure: before contract compliance on the business logic can be evaluated, the foundational scaffold is already incomplete.

Failure mode 2: core logic exists, but contract-level edge cases remain broken: `codex/gpt-5.3-codex` is noticeably stronger on the happy path. It gets registration, login, and current-user retrieval working, which means substantial parts of the backend are correctly wired. Yet it still fails because greenfield tasks test more than just basic route success. Missing CORS headers violate the HTTP contract, omitted directories violate the repository contract, and accepting forbidden username updates violates the API contract. The self-unfollow failure further shows that relation-management edge cases remain underspecified in the generated code.

Failure mode 3: auditing is treated as optional, but the benchmark treats it as first-class: `codex/gpt-5.5` and `gemini-cli/gemini-3.1-pro-preview` reveal another important benchmark property. Both appear to implement enough route logic that several endpoints respond plausibly, but they fail because the audit trail does not evolve as required. This is especially revealing in a greenfield setting: a model may prioritize visible endpoint behavior and underweight non-functional instrumentation, whereas the benchmark explicitly evaluates auditing as part of correctness.

What this example shows about the dataset: `task-rw-9` illustrates why greenfield implementation tasks are a necessary complement to incremental-editing tasks. The difficulty is not only in implementing endpoints from documentation; it is also in creating the surrounding project skeleton, middleware configuration, and audit instrumentation that make those endpoints production-like. The results show that current coding agents can often produce a believable subset of the system, but they still struggle to satisfy all layers of the specification simultaneously. That makes `task-rw-9` a strong qualitative example of realistic backend bootstrapping under benchmark constraints.

D.3. Bug-fixing example: `task-rw-9_2`

Finally, we consider `task-rw-9_2`, a *bug-fixing* task built on top of the same RealWorld slice as `task-rw-9`. Unlike the greenfield task, this one assumes that most of the backend already exists and asks the agent to repair two specific correctness issues while preserving existing functionality. This makes it a useful complement to the earlier examples because it stresses whether a model can perform a narrow maintenance change in a system that already mostly works.

Input given to the agent

Task

Update the existing implementation to fix the reported issues.

Refer to the complete technical specification in the `api.md` file and preserve the existing API contracts while applying the fixes.

Fixes to apply

- `PUT /api/user` must reject username updates.
- Failed lookups for non-existent users must still be audited correctly.

APIs to be considered in this task

- `POST /api/users/login`
- `POST /api/users`
- `GET /api/user`
- `PUT /api/user`
- `GET /api/profiles/:username`
- `POST /api/profiles/:username/follow`
- `DELETE /api/profiles/:username/follow`
- `GET /api/audits`
- `GET /hello`

The hidden tests show that this task is not merely about rejecting one field in `PUT /api/user`. It also checks whether the surrounding audit subsystem remains semantically correct for failed profile-related operations. In effect, the task evaluates whether an agent can make a localized patch while respecting a cross-cutting non-functional contract.

Expected output and what the tests check: The hidden verifier contains eight tests. The two most important bug-fix targets are:

- **Forbidden username updates:** PUT /api/user must reject attempts to change username, while still allowing valid updates such as bio, image, and password.
- **Audit preservation on failed profile operations:** failed lookups and invalid profile actions involving non-existent users or self-directed follow operations must still update the audit log in the way expected by the verifier.

This is what makes the task interesting: the functional fix is small, but part of correctness lies in instrumentation rather than endpoint return values alone.

How the five evaluated agent/model pairs perform. Table 6 summarizes the verifier outcomes. All five pairs end up with the same aggregate score, but for slightly different reasons.

Table 6. Performance of five agent/model pairs on task-rw-9_2. “Passed tests” comes directly from the verifier logs for the eight hidden tests.

Agent/model	Passed tests	Primary failure mode
claude-code/ claude-opus-4.7	6/8	Rejects forbidden username updates, but misses the required audit increments for failed username-change and self-follow operations.
claude-code/ claude-sonnet-4.6	6/8	Same broad pattern as Opus: the main API behavior is mostly correct, but failed operations are not audited as strictly as required.
codex/ gpt-5.3-codex	6/8	Also reaches the right endpoint behavior on the main paths, yet omits the expected audit updates for failed username-change and self-follow cases.
codex/ gpt-5.5	6/8	Fixes the user-update contract, but still misses audit updates on failed username-change and non-existent-profile lookup paths.
gemini-cli/ gemini-3.1-pro-preview	6/8	Shows the same high-level pattern as codex/ gpt-5.5: the routes mostly work, but audit bookkeeping remains incomplete on negative-path profile operations.

The most striking result is the *uniformity* of the outcomes. Every evaluated agent/model pair reaches 6/8, which suggests that the benchmark is isolating a failure mode that is systematically hard for current systems.

Failure mode 1: business logic fixed, but failed-action auditing omitted: claude-code/claude-sonnet-4.6, claude-code/claude-opus-4.7, and codex/gpt-5.3-codex all largely solve the visible functional problem. In particular, they no longer fail because username updates are accepted. However, the verifier shows that they still under-handle audit updates on rejected operations such as failed username changes or self-follow attempts. This is a classic maintenance-task pitfall: the explicit surface bug is fixed, but a secondary contract remains unmet.

Failure mode 2: partial audit awareness, but still incomplete on negative paths: codex/gpt-5.5 and gemini-cli/gemini-3.1-pro-preview exhibit a closely related pattern, but the missed audit cases are slightly different. Their remaining failures are tied to incomplete bookkeeping for rejected operations involving username updates and non-existent profile lookups. Again, the main API behavior is close to correct; the problem is that the instrumentation layer is still not aligned with the specification.

What this example shows about the dataset: task-rw-9_2 is a useful bug-fixing example because it highlights a subtle but important point: in realistic software maintenance, correctness is often defined not only by outward-facing route behavior, but also by side effects such as logging, auditing, and security instrumentation. The results show that current coding agents can often repair the obvious functional defect, yet still miss the surrounding operational contract. That makes task-rw-9_2 a strong example of why maintenance-oriented benchmark tasks should include hidden checks for non-functional correctness as well as API behavior.

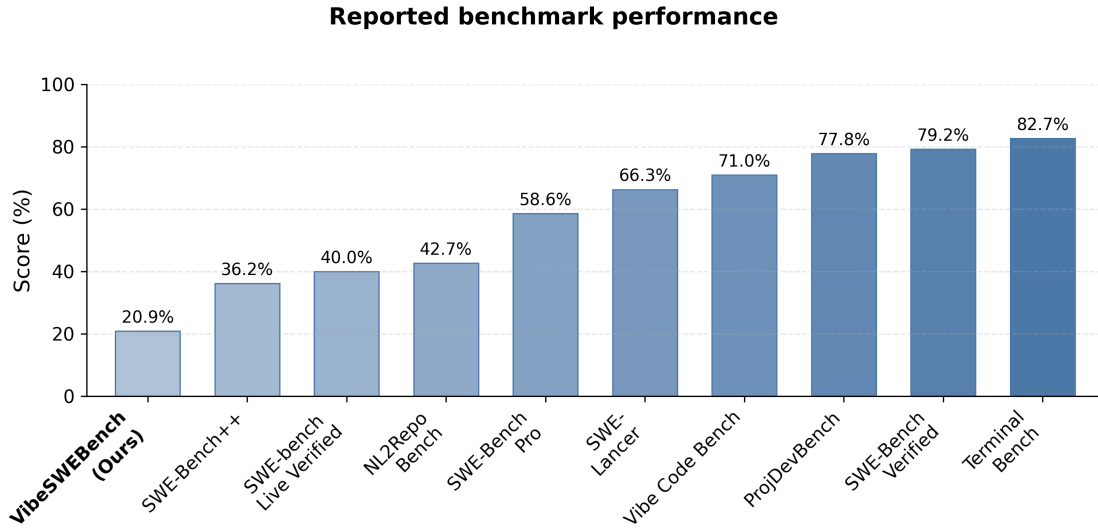


Figure 5. Maximum score received across different benchmark. All the sources of this result are mentioned in Appendix E.

E. Benchmark Result Sources

The following links were used as the sources for the reported benchmark values in the histogram shown in Figure 5.

- **Terminal Bench** — 82.7
Source: <https://openai.com/index/introducing-gpt-5-5/>
- **SWE-Verified** — 79.20
Source: <https://www.swebench.com/>
- **ProjDevBench** — 77.85
Source: <https://arxiv.org/pdf/2602.01655>
- **Vibe Code Bench** — 71.00
Source: <https://www.vals.ai/benchmarks/vibe-code>
- **SWE-Lancer** — 66.3
Source: <https://llm-stats.com/benchmarks/swe-lancer>
- **SWE-Bench Pro** — 58.6
Source: <https://openai.com/index/introducing-gpt-5-5/>
- **NL2Repo-Bench** — 42.7
Source: <https://llm-stats.com/benchmarks/nl2repo>
- **SWE-bench Live Verified** — 40.00
Source: <https://swe-bench-live.github.io/>
- **SWE-BENCH++** — 36.20
Source: <https://arxiv.org/pdf/2512.17419>

F. Limitations and Future Work

While **VIBESWEBENCH** provides a robust framework for evaluating vibe coding agents, we acknowledge several inherent limitations and identify concrete directions for future work. The current version of **VIBESWEBENCH** covers only backend components; a complete vibe coding benchmark must also assess an agent’s ability to produce correct, specification-compliant user interfaces. Extending the benchmark to include frontend tasks—requiring comparison of agent-generated UIs against reference designs—is our highest priority for future work, as it is essential for a holistic assessment of the full

implementation step. Second, the dataset currently focuses exclusively on Node.js backend projects, leaving **multi-language coverage** as a significant gap. Real-world vibe coding spans a diverse ecosystem of backend technologies, including Java Spring Boot, Go Gin, Rust Axum, and PHP Laravel, and future versions of the benchmark should include tasks across these stacks to ensure that evaluation results generalize beyond a single language paradigm. Third, **VIBESWEBENCH** will include DevOps problems to push the frontier to be good at deployment workflow. Fourth, **VIBESWEBENCH** targets only the implementation step of the vibe coding workflow (step 2). **Evaluating step 1**—the requirement evolution phase, in which an agent assists a user in formalizing and standardizing their requirements into a precise specification—is an equally important and currently unmeasured capability, and developing a dataset and evaluation methodology for this phase represents a substantial open research direction. Finally, **automated test case generation** is a prerequisite for scaling the benchmark to a larger and more diverse collection of projects. Currently, test cases are authored manually for each task, which is labour-intensive and limits the rate at which new projects can be incorporated. Developing reliable methods to automatically synthesize correct, comprehensive test cases from a reference specification—without human intervention—would substantially reduce this bottleneck and is a key enabler for long-term benchmark growth.

G. Broader Impacts

VIBESWEBENCH has the potential to produce positive impact by enabling more rigorous and realistic evaluation of autonomous coding agents. By centering evaluation on formal specifications and deterministic test suites, the benchmark encourages the development of systems that are more reliable, auditable, and aligned with user intent. We expect this to benefit both researchers and practitioners by making it easier to diagnose concrete failure modes—such as specification misreading, incomplete implementation, and regression-inducing edits—that are currently obscured by less grounded evaluation protocols. More broadly, improved evaluation may support safer deployment of coding agents in real software engineering workflows, especially in settings where correctness and maintainability matter.

At the same time, the benchmark may also have negative or dual-use implications. Better evaluation can accelerate the capabilities of coding agents, including systems that could be misused to automate the development of harmful or insecure software. In addition, public benchmarks can create incentives for overfitting, benchmark gaming, or contamination through inclusion in training corpora, which may inflate apparent progress without corresponding real-world improvements. There is also a risk that strong performance on benchmark tasks could be misinterpreted as evidence that agents are ready for autonomous production use, despite the substantial capability gaps identified in our experiments.

We take several steps to mitigate these risks. First, evaluation is grounded in hidden, deterministic test suites rather than subjective judgment, reducing the opportunity for superficial optimization. Second, as discussed in last paragraph of Section 3.3, we withhold parts of the evaluation artifacts and inject a benchmark canary string to discourage training-data contamination. Third, we frame **VIBESWEBENCH** as a diagnostic tool for measuring failure modes and limitations, not as a certification of deployment readiness. We hope this benchmark supports more transparent and responsible progress in autonomous software engineering.